



## PROJECT INFORMATION

Project title : Smart\_Daily\_Expense\_Tracker

Course title : Introduction To Problem Solving And programming

Course code : CSE 1021

Course type : Flipped course

Course credits : 4

Professor : Dr.Senthilkumar R

Slot : B11+B12+B13+C14+E11+E12

Submitted by : Saraniesh Murugesh

Registration number : 26BAI10923

Submission date : 30/09/26

# VITYARTHI – PROJECT

## INTRODUCTION :

Students may make several small purchases during a day — food, transport, stationery, subscriptions — and when these expenses are recorded informally, or not recorded at all, it can become difficult to determine where money is being spent or how much has been spent over a period. Over a month, small untracked purchases can add up to a significant and easily overlooked amount.

This project provides a simple terminal-based system in which expenses are entered in a consistent format and later retrieved or analysed. Rather than relying on a spreadsheet or a paper notebook, the user interacts with a menu-driven program that validates each entry and stores it automatically.

The application is intentionally lightweight and uses Python's standard library, making it suitable for demonstrating programming fundamentals in a first-year academic project. No external frameworks are required to install or run it, which keeps the project easy to set up, review and grade.

### *Key characteristics of the system*

- Menu-driven interaction
- Local persistence
- Input validation
- Expense analysis
- Modular source organisation

## PROBLEM STATEMENT :

Students and individuals often record daily spending informally, making it difficult to identify spending patterns. A small structured application can reduce this difficulty by keeping records in one place and calculating summaries automatically, instead of requiring the user to manually total amounts at the end of each week or month.

# **VITYARTHI – PROJECT**

The proposed system accepts a category, amount and date for each expense. It stores the record, allows retrieval and deletion, and converts raw records into useful summaries such as category totals, daily totals and the single highest-spending category.

The project focuses on a clearly defined academic scope rather than attempting to reproduce a full commercial finance application. The goal is correctness and clarity of implementation rather than breadth of features.

## **PROJECT OBJECTIVES :**

The main objective is to develop an understandable and reliable Python program for daily expense tracking. Additional objectives are to apply programming concepts in a complete project workflow, from requirements through design, implementation and testing.

- Record category, amount and date.
- Validate input before storage.
- Persist records using JSON.
- Search and view records.
- Generate category and daily summaries.
- Calculate total spending.
- Identify the highest-spending category.
- Demonstrate modular programming and automated testing.

Together, these objectives ensure that the finished project is not just a working script, but a small, well-organised piece of software that follows recognisable engineering practice.

## **SCOPE OF THE PROJECT :**

The current project covers local single-user expense tracking through a command-line interface. It focuses on recording, retrieving and

# VITYARTHI – PROJECT

analysing basic expense records, using only the Python standard library for storage and computation.

Authentication, cloud synchronisation, advanced graphical dashboards and multi-user management are outside the current scope. These are listed separately under future enhancements, since they would require additional infrastructure beyond what a single-semester course project is expected to deliver.

- In scope: local records
- In scope: terminal interface
- In scope: summaries
- Out of scope: cloud storage
- Out of scope: multi-user authentication
- Out of scope: GUI dashboard

## FUNCTIONAL REQUIREMENTS :

The functional requirements define the operations available to the user through the terminal menu.

- Add and validate an expense.
- View stored expenses.
- Search by category.
- Delete a selected expense.
- Calculate total spending.
- Generate category-wise summary.
- Generate daily summary.
- Identify highest-spending category.
- Save and reload data using JSON.
- Handle invalid input without terminating the application.

# VITYARTHI – PROJECT

## NON-FUNCTIONAL REQUIREMENTS :

Non-functional requirements describe qualities expected from the system, beyond the specific operations it performs.

- Usability: simple menu-driven interaction.
- Reliability: validation and exception handling.
- Maintainability: separated modules and functions.
- Performance: lightweight processing for normal daily records.
- Resource efficiency: standard-library implementation.
- Portability: compatible Python environment.
- Testability: repeatable automated checks.

## REQUIREMENTS TRACEABILITY MATRIX :

The table below maps each functional requirement to the module primarily responsible for implementing it, and to the test case that verifies it. This traceability view helps confirm that every requirement is both implemented and independently tested.

| Req. ID | Requirement                 | Responsible module                | Verified by                          |
|---------|-----------------------------|-----------------------------------|--------------------------------------|
| FR-1    | Add and validate an expense | expense_manager.py, validators.py | Valid category / amount / date tests |
| FR-2    | View stored expenses        | menu.py, expense_manager.py       | Manual run / view menu option        |
| FR-3    | Search by category          | expense_manager.py                | Search menu option                   |
| FR-4    | Delete a selected expense   | expense_manager.py                | Invalid deletion number test         |
| FR-5    | Calculate total spending    | reports.py                        | Total spending test                  |
| FR-6    | Category-wise summary       | reports.py                        | Category summary test                |
| FR-7    | Daily summary               | reports.py                        | Daily summary test                   |
| FR-8    | Highest-spending category   | reports.py                        | Highest category test                |
| FR-9    | Save/reload via JSON        | storage.py                        | Startup reload / manual run          |

# VITYARTHI – PROJECT

| Req. ID | Requirement          | Responsible module     | Verified by                          |
|---------|----------------------|------------------------|--------------------------------------|
| FR-10   | Handle invalid input | menu.py, validators.py | Invalid amount / date / choice tests |

# **VITYARTHI – PROJECT**

## **SYSTEM ARCHITECTURE :**

The application separates user interaction, business logic, validation, reporting and persistence. The menu receives commands, ExpenseManager coordinates operations, validators check input, reports calculate summaries, and storage handles JSON data.

This separation allows individual components to be changed or tested without rewriting the complete application. For example, the storage format could later be changed from JSON to SQLite without affecting the menu or reporting logic, since both communicate with storage only through a small, well-defined interface.

- User → Menu → ExpenseManager
- ExpenseManager → Validators / Reports / Storage
- Storage → JSON data file
- Utilities support formatting and display.

## **USE CASE ANALYSIS :**

The main actor is the Student/User. The user interacts with the system through the terminal menu, and each menu option represents a use case. Every use case below begins at the main menu and returns to it once the operation is complete.

- Add Expense
- View Expenses
- Search by Category
- Category-wise Summary
- Daily Summary
- Highest Spending Category
- Delete Expense
- Total Spending

# VITYARTHI – PROJECT

- Exit

## DETAILED WORKFLOW :

The application loads saved data at startup and then displays the menu. The selected operation is processed and the result is displayed. Changes such as adding or deleting an expense are persisted to the JSON file immediately, so that no work is lost if the program is closed unexpectedly.

For an add operation, the system validates category, amount and date before appending the record. If any field fails validation, the user is asked to re-enter that field rather than having the entire operation cancelled.

- Start
- Load saved data
- Display menu
- Read choice
- Validate/process request
- Display result
- Save changes when required
- Return to menu
- Exit

## SEQUENCE OF OPERATIONS :

A typical add-expense interaction begins with the user entering data through the menu. ExpenseManager receives the request and calls the validation functions. After successful validation, the record is appended and storage writes the updated data.

- User → Menu
- Menu → ExpenseManager

# VITYARTHI – PROJECT

- ExpenseManager → Validators
- ExpenseManager → Storage
- Storage → JSON
- Result → Menu → User

# **VITYARTHI – PROJECT**

## **MODULE AND COMPONENT DESIGN :**

The project is divided into focused Python files so that each module has a clear, single responsibility. This makes the codebase easier to navigate, test and extend.

- `main.py` — application entry point; loads data and starts the menu loop.
- `menu.py` — terminal menu and user interaction; reads choices and delegates to `ExpenseManager`.
- `expense_manager.py` — core expense operations: add, view, search, delete.
- `validators.py` — input validation for category, amount and date.
- `storage.py` — JSON loading and saving.
- `reports.py` — summaries and analysis: totals, category and daily breakdowns.
- `utils.py` — formatting helpers used by the menu and reports.
- `tests/test_expense_tracker.py` — automated tests covering validation and reporting.

Each module exposes a small number of functions with a clear purpose, which keeps individual files short and makes it straightforward to locate the code responsible for any given menu option.

## **DATA MODEL AND STORAGE DESIGN :**

Each expense is represented as a three-field tuple: category, amount and date. A list stores the collection of expenses in memory during execution. JSON is used for persistence because it is simple, human-readable and available through Python's standard library without any extra dependency.

The stored representation contains the same three pieces of information and is reloaded when the program starts, so that the in-memory list and

# VITYARTHI – PROJECT

the JSON file always stay in agreement after every save operation. A single saved record is illustrated below.

```
{  
  "category": "Food",  
  "amount": 150.0,  
  "date": "21-09-2026"  
}
```

The complete data file is a JSON array of such objects, one per recorded expense, making it easy to inspect or back up manually if required.

## INPUT VALIDATION AND ERROR HANDLING :

Validation prevents incorrect data from entering the stored dataset. Category input must not be empty, amount must be numeric and positive, and date must follow DD-MM-YYYY and represent a valid calendar date.

Exceptions are caught by the menu layer so the user receives an error message instead of the application terminating unexpectedly. This keeps a single bad entry from disrupting an entire session.

- Empty category → rejected
- Negative or non-numeric amount → rejected
- Invalid date → rejected
- Invalid menu choice → explanatory message
- Invalid deletion number → rejected

# VITYARTHI – PROJECT

## CORE ALGORITHM LOGIC :

This section describes, at a conceptual level, the logic behind the three most important computations in the system: input validation, category aggregation and highest-category identification. The notation below is illustrative pseudocode rather than literal source code.

### *Amount validation*

```
function is_valid_amount(value):  
    if value is numeric AND value > 0:  
        return True  
    else:  
        return False
```

### *Category-wise aggregation*

```
totals = empty dictionary  
for each expense in expense_list:  
    key = expense.category  
    totals[key] = totals.get(key, 0) + expense.amount
```

### *Highest-spending category*

```
highest_category = key in totals  
                    with the maximum totals[key]
```

These three routines are combined by ExpenseManager and reports.py to turn a flat list of expense records into validated entries and, later, into the category, daily and total summaries described in the next section.

## REPORTING AND ANALYSIS LOGIC :

The reports module converts raw records into useful information. Category totals are accumulated in a dictionary keyed by category, while daily totals use dates as dictionary keys.

The highest-spending category is obtained from the category summary, and total spending is the sum of all expense amounts. A short worked example is shown below.

| Category | Entries    | Category total |
|----------|------------|----------------|
| Food     | ₹100 + ₹50 | ₹150           |

# VITYARTHI – PROJECT

| Category      | Entries | Category total |
|---------------|---------|----------------|
| Travel        | ₹200    | ₹200           |
| Overall total | —       | ₹350           |

In this example, Food is the highest-spending category since ₹150 is greater than the ₹200 travel total is not — Travel would in fact be reported as the highest category, illustrating why the comparison is performed on the aggregated totals rather than on individual entries.

## IMPLEMENTATION AND PYTHON CONCEPTS :

The project applies lists, tuples, dictionaries, loops, conditions, functions, modules, exceptions and JSON handling. These concepts work together to create a complete application rather than isolated code examples.

- Lists
- Tuples
- Dictionaries
- Loops
- Conditional statements
- Functions and parameters
- Modules and imports
- Exception handling
- JSON file handling
- Automated testing

Using these fundamentals together, rather than in isolation, was a deliberate choice: it mirrors how introductory concepts are actually combined in real, working software.

# VITYARTHI – PROJECT

## TESTING STRATEGY AND TEST CASES :

The automated tests focus on validation and reporting logic. They include normal cases and invalid inputs. The final report should include the student's own test execution output.

| Test case        | Input         | Expected result               |
|------------------|---------------|-------------------------------|
| Valid category   | "Food"        | Accepted                      |
| Invalid amount   | -50 or "abc"  | Rejected                      |
| Valid date       | 21-09-2026    | Accepted                      |
| Invalid date     | 31-02-2026    | Rejected                      |
| Category summary | Mixed entries | Correct totals per category   |
| Daily summary    | Mixed entries | Correct totals per date       |
| Highest category | Mixed entries | Correct top category returned |
| Total spending   | Mixed entries | Correct overall sum           |

## SAMPLE OUTPUT AND SCREENSHOT PLAN :

The report should use screenshots from the student's own execution. These provide evidence that the implemented functions work as described. The table below lists each screenshot that should be captured and what it is meant to demonstrate.

| Screenshot                 | Purpose                                           |
|----------------------------|---------------------------------------------------|
| Main menu                  | Shows all available options at startup.           |
| Adding an expense          | Shows validated input being accepted and saved.   |
| Viewing expenses           | Shows stored records listed to the user.          |
| Category-wise summary      | Shows totals grouped by category.                 |
| Daily summary              | Shows totals grouped by date.                     |
| Highest category and total | Shows the two headline summary figures.           |
| Search / delete            | Shows retrieval and removal of a specific record. |
| Automated test results     | Shows the test suite passing.                     |

*Actual runtime output screenshot is included on the next page.*

# SMART DAILY EXPENSE TRACKER

## SAMPLE RUNTIME OUTPUT / EXECUTION SCREENSHOT

The screenshot below shows the actual terminal execution of the submitted application. It demonstrates the main menu and successful addition of expense records.

```
C:\Program Files\WindowsApps\PythonSoftwareFoundation.PythonManager_26.3.240.0_x64_3847v3x7pw1km\python.exe

=====
SMART DAILY EXPENSE TRACKER
=====
1. Add Expense
2. View All Expenses
3. Search by Category
4. Category-wise Summary
5. Daily Summary
6. Show Highest Spending Category
7. Delete Expense
8. Show Total Spending
9. Exit
Enter your choice: 1
Enter category: car
Enter amount: 1000000
Enter date (DD-MM-YYYY): 02-02-2018
Expense added successfully!

=====
SMART DAILY EXPENSE TRACKER
=====
1. Add Expense
2. View All Expenses
3. Search by Category
4. Category-wise Summary
5. Daily Summary
6. Show Highest Spending Category
7. Delete Expense
8. Show Total Spending
9. Exit
Enter your choice: 1
Enter category: food
Enter amount: 4000
Enter date (DD-MM-YYYY): 02-02-2018
Expense added successfully!

=====
SMART DAILY EXPENSE TRACKER
=====
```

**Figure: Console output during expense entry**

The displayed run records entries under the categories "car" and "food" and confirms successful saving.

# **VITYARTHI – PROJECT**

## **DESIGN DECISIONS :**

The implementation choices were made to keep the project understandable while demonstrating project-level organisation.

- Modules separate responsibilities.
- Tuples represent the three-field record.
- Dictionaries support aggregation.
- JSON provides simple persistence.
- CLI avoids unnecessary external dependencies.
- Validation is reusable.
- Tests provide repeatable checking.

## **ASSUMPTIONS AND CONSTRAINTS :**

A few simplifying assumptions were made to keep the project within the scope of a first-semester course, while still producing a genuinely usable tool.

- A single user runs the application on a single machine at a time.
- Amounts are recorded in a single currency, without conversion.
- Dates are entered manually in DD-MM-YYYY format rather than picked from a calendar widget.
- The JSON data file is trusted; the application does not defend against a manually corrupted file.
- The program is run from a terminal with a standard Python 3 interpreter available.

## **CHALLENGES AND SOLUTIONS :**

Input validation is necessary because users can enter unexpected values. The solution is to validate before storage, rejecting bad input at the point of entry rather than trying to clean it up later.

# VITYARTHI – PROJECT

Persistence is required so data is not lost when the program closes. JSON provides a simple local solution that needs no external database engine.

As features increase, a single large file becomes harder to maintain. Separating modules keeps responsibilities clear and makes it possible to test each part of the system on its own.

## PROJECT TIMELINE AND WORK BREAKDOWN :

The project was carried out in stages, moving from requirements to a working, tested application. The approximate breakdown of effort is shown below.

| Stage | Activity                                                   |
|-------|------------------------------------------------------------|
| 1     | Requirements gathering and problem definition              |
| 2     | Architecture and module design                             |
| 3     | Core implementation (validators, storage, expense manager) |
| 4     | Reporting and analysis logic                               |
| 5     | Menu integration and manual testing                        |
| 6     | Automated test writing and execution                       |
| 7     | Report documentation and screenshot collection             |

## LEARNINGS AND OUTCOMES :

The project provides practical experience in converting a problem into a structured Python application and connecting user input, processing, storage and output.

- Modular Python programming
- Lists, tuples and dictionaries
- Validation and exception handling
- JSON persistence
- Summary calculations

# VITYARTHI – PROJECT

- Automated testing
- Project documentation

## **LIMITATIONS AND FUTURE ENHANCEMENTS :**

The current application is a local command-line project. It does not include cloud synchronisation, multi-user accounts or graphical dashboards.

- GUI application
- Web/mobile interface
- Monthly and yearly reports
- Charts
- Budget limits and alerts
- CSV export
- SQLite database
- Authentication
- Cloud backup

## **CONCLUSION :**

The Smart Daily Expense Tracker demonstrates how an everyday problem can be addressed with a structured Python application. It combines input handling, validation, data structures, modular programming, persistence, reporting and testing.

The current version provides a clear foundation for future extensions while remaining suitable for an introductory academic project.

# VITYARTHI – PROJECT

## GLOSSARY OF TERMS :

| Term        | Meaning                                                      |
|-------------|--------------------------------------------------------------|
| Category    | A label describing the type of expense, e.g. Food, Travel.   |
| Persistence | Storing data so it survives after the program closes.        |
| JSON        | JavaScript Object Notation; a text-based data format.        |
| Validation  | Checking that input meets required rules before use.         |
| Aggregation | Combining many records into a single summary value.          |
| CLI         | Command-Line Interface; a text-based way of using a program. |
| Module      | A single Python file grouping related functions.             |

## USER MANUAL – HOW TO RUN THE APPLICATION :

The following steps describe how to set up and run the Smart Daily Expense Tracker from a terminal.

1. Install Python 3 if it is not already available on the system.
2. Open a terminal and navigate to the project folder.
3. Run the application with: `python main.py`
4. Choose an option from the numbered menu, such as adding or viewing an expense.
5. Follow the prompts to enter category, amount and date when adding an expense.
6. Select the exit option to close the application; all changes are already saved to disk.

To run the automated test suite instead of the application, use: `python -m unittest discover tests`

# **VITYARTHI – PROJECT**

## **REFERENCES AND SUBMISSION CHECKLIST :**

The implementation uses Python standard-library concepts and course learning context. Before submission, all project details and runtime behaviour should be verified.

- Python standard library documentation.
- CSE1012 course learning materials.
- Student-created implementation and test cases.
- Run every menu option.
- Run automated tests.
- Add screenshots from your own execution.
- Verify student details and date.
- Check the final ZIP contents.